QuillAudits

AUDIT REPORT

June 2026

For

Zynk

Table of Content

Executive Summary

Project Name	Zynk Labs
Project URL	https://www.zynk.money/
Overview	Zynk Orbit is an Anchor-based Solana smart contract that acts as a permissioned token routing layer, moving tokens from external signers and vault PDAs into a central wallet (ZOV), and disbursing them to whitelisted beneficiaries. It uses a hardcoded, three-role privilege model and a single Record PDA as the backbone of its accounting.
Audit Scope	The scope of this Audit was to analyze the Zynk Labs Smart Contracts for quality, security, and correctness.
Source Code link	https://github.com/Zynklabs/zynk-protocol-solana-programs/blob/zynk-orbit-v1/programs/zynk-orbit/src/lib.rs
Branch	zynk-orbit-v1
Contracts in Scope	lib.rs
Commit Hash	beebd5e51484b39f2e8a285ab032bc93ed29eb09
Language	Rust
Blockchain	Solana
Method	Manual Analysis, Functional Testing, Automated Testing
Review 1	10th Jun 2026 - 13th Jun 2026
Review 2	13th June 2026 - 17th June 2026
Fixed In	350a98e2dcac8cb140ee094604f0f64a5c2021a0

Verify the Authenticity of Report on QuillAudits Leaderboard:

https://www.quillaudits.com/leaderboard

Number of Issues per Severity

Critical	0 (0.0%)
High	0 (0.0%)
Medium	0 (0.0%)
Low	3 (50.0%)
Informational	3 (50.0%)

Severity

Issues	Critical	High	Medium	Low	Informational
Open	0	0	0	0	0
Acknowledged	0	0	0	0	0
Partially Resolved	0	0	0	0	0
Resolved	0	0	0	**3**	**3**

Summary of Issues

Issue No.	Issue Title	Severity	Status
1	collect() lacks settlement record initialization and record-domain separation	Low	Resolved
2	Deposit events do not enforce accounting-grade uniqueness or value invariants	Low	Resolved
3	Privileged settlement and revoke operations do not emit lifecycle events	Low	Resolved
4	Unused manual account-close helper should be removed	Informational	Resolved
5	Unnecessary account requirements and writable metas add avoidable account surface	Informational	Resolved
4	revoke() can close any Orbit `Record` supplied by admin	Informational	Resolved

Checked Vulnerabilities

We have scanned the solana program for commonly known and more specific vulnerabilities. Here are some of the commonly known vulnerabilities that we considered:

- Centralization of control
- Ether theft
- Improper or missing events
- Logical issues and flaws
- Arithmetic Computations Correctness
- Race conditions/front running
- SWC Registry
- Re-entrancy
- Timestamp Dependence
- Gas Limit and Loops
- Exception Disorder
- Gasless Send
- Use of tx.origin
- Malicious libraries
- Compiler version not fixed
- Address hardcoded
- Divide before multiply
- Integer overflow/underflow
- ERC's conformance
- Dangerous strict equalities

- [x] Tautology or contradiction
- [x] Return values of low-level calls
- [x] Missing Zero Address Validation
- [x] Private modifier
- [x] Revert/require functions
- [x] Multiple Sends
- [x] Using suicide
- [x] Using delegatecall
- [x] Upgradeable safety
- [x] Using throw
- [x] Using inline assembly
- [x] Style guide violation
- [x] Unsafe type inference
- [x] Implicit visibility level

Techniques and Methods

Throughout the audit of Solana Programs, care was taken to ensure:

- The overall quality of code
- Use of best practices
- Code documentation and comments, match logic and expected behavior
- Token distribution and calculations are as per the intended behavior mentioned in the whitepaper
- Efficient use of gas
- Code is safe from re-entrancy and other vulnerabilities

The following techniques, methods, and tools were used to review all the smart contracts:

Structural Analysis

In this step, we have analysed the design patterns and structure of Solana programs. A thorough check was done to ensure the Solana program is structured in a way that will not result in future problems.

Static Analysis

Static analysis of Solana programs was done to identify contract vulnerabilities. In this step, a series of automated tools are used to test the security of Solana programs.

Code Review / Manual Analysis

Manual Analysis or review of code was done to identify new vulnerabilities or verify the vulnerabilities found during the static analysis. Contracts were completely manually analysed, and their logic was checked and compared with the one described in the whitepaper. Besides, the results of the automated analysis were manually verified.

Gas Consumption

In this step, we have checked the behaviour of Solana programs in production. Checks were done to know how much gas gets consumed and the possibilities of optimising code to reduce gas consumption.

Types of Severity

Every issue in this report has been assigned to a severity level. There are five levels of severity, and each of them has been explained below.

Critical: Immediate and Catastrophic Impact

Critical issues are the ones that an attacker could exploit with relative ease, potentially leading to an immediate and complete loss of user funds, a total takeover of the protocol's functionality, or other catastrophic failures. Critical vulnerabilities are non-negotiable; they absolutely must be fixed.

High (H): Significant Risk of Major Loss or Compromise

High-severity issues represent serious weaknesses that could result in significant financial losses for users, major malfunctions within the protocol, or substantial compromise of its intended operations. While exploiting these vulnerabilities might require specific conditions to be met or a moderate level of technical skill, the potential damage is considerable. These findings are critical and should be addressed and resolved thoroughly before the contract is put into the Mainnet.

Medium (M): Potential for Moderate Harm Under Specific Circumstances

Medium-severity bugs are loopholes in the protocol that could lead to moderate financial losses or partial disruptions of the protocol's intended behavior. However, exploiting these vulnerabilities typically requires more specific and less common conditions to occur, and the overall impact is generally lower compared to high or critical issues. While not as immediately threatening, it's still highly recommended to address these findings to enhance the contract's robustness and prevent potential problems down the line.

Low (L): Minor Imperfections with Limited Repercussions

Low-severity issues are essentially minor imperfections in the smart contract that have a limited impact on user funds or the core functionality of the protocol. Exploiting these would usually require very specific and unlikely scenarios and would yield minimal gain for an attacker. While these findings don't pose an immediate threat, addressing them when feasible can contribute to a more polished and well-maintained codebase.

Informational (I): Opportunities for Improvement, Not Immediate Risks

Informational findings aren't security vulnerabilities in the traditional sense. Instead, they highlight areas related to the clarity and efficiency of the code, gas optimization, the quality of documentation, or adherence to best development practices. These findings don't represent any immediate risk to the security or functionality of the contract but offer valuable insights for improving its overall quality and maintainability. Addressing these is optional but often beneficial for long-term health and clarity.

Severity Matrix

Impact

Likelihood	High	Medium	Low
High	Critical	High	Medium
Medium	High	Medium	Low
Low	Medium	Low	Low

Impact

- **High** - leads to a significant material loss of assets in the protocol or significantly harms a group of users.
- **Medium** - only a small amount of funds can be lost (such as leakage of value) or a core functionality of the protocol is affected.
- **Low** - can lead to any kind of unexpected behavior with some of the protocol's functionalities that's not so critical.

Likelihood

- High - attack path is possible with reasonable assumptions that mimic on-chain conditions, and the cost of the attack is relatively low compared to the amount of funds that can be stolen or lost.
- Medium - only a conditionally incentivized attack vector, but still relatively likely.
- Low - has too many or too unlikely assumptions or requires a significant stake by the attacker with little or no incentive.

Low Severity Issues

collect() lacks settlement record initialization and record-domain separation

Resolved

Path
lib.rs

Function Name
`collect()`

Description
depositRewardPool() uses a before/after balance comparison to detect fee-on-transfer tokens, reverting if the received amount differs from what was requested. stake(), which pulls the same staking token inward, performs no equivalent check and immediately credits the full requested amount to the user's position and the global total. If the staking token carries a transfer fee, every stake would overstate the contract's actual holdings. The accounting shortfall would compound silently and eventually cause withdrawals to fail. Since stakingToken is immutable, the risk is bounded to the token configured at deployment, but the inconsistency within the same contract is a design quality gap.

The same Record structure is also used across whitelist and settlement-style logic without an explicit record type, record domain, mint binding, or settlement-specific PDA constraint.

Impact
This is not a public attacker exploit under the current trust model because collect and disburse are manager-gated. The practical impact is operational correctness: if the settlement flow uses whitelist-created records, nonzero collect calls cannot complete. If settlement depends on pre-existing finite records, the current scoped program does not show how those records are initialized.

Likelihood
Low. The issue is reachable through manager-controlled settlement flows or an undefined external finite-record lifecycle and does not provide a direct public attacker path.

Recommendation
Create a dedicated settlement record type or add explicit record-domain fields and PDA seed constraints. collect should initialize or require the correct finite settlement record, and disburse should only accept records from that settlement domain with the expected mint and recipient binding.

Low Severity Issues

Deposited event lacks accounting-grade uniqueness and destination specificity

Resolved

Path
lib.rs

Function Name
`deposit()`

Description
The deposit instruction is publicly callable by any signer that has a matching whitelist Record PDA. The instruction accepts a caller-supplied request_id, does not reject amount == 0, and emits a Deposited event after the SPL token transfer. The event emits to = destination_token_account.owner, which is the fixed ZOV authority, rather than the actual destination token account address.

Impact
A whitelisted caller can emit successful duplicate or zero-value deposit events. The caller can also deposit into different same-mint token accounts owned by the ZOV authority while the event still reports the same to value. If an external consumer treats the event as authoritative without checking transaction idempotency, actual destination token account, and token balance deltas, this can create accounting or reconciliation errors.

Likelihood
Low. The event behavior is publicly reachable by whitelisted signers, but higher financial impact depends on unsafe off-chain consumption.

Recommendation
Reject zero-amount deposits before the SPL token CPI. If request_id is expected to be unique, enforce idempotency on-chain with a request PDA or clearly document that it is informational only. Emit the actual destination token account address, or rename the current to field to to_owner and add a destination_token_account field.

Low Severity Issues

Privileged settlement and revoke operations do not emit lifecycle events

Resolved

Path

lib.rs

Function Name

`collect(), disburse(), revoke()`

Description

The deposit instruction emits a Deposited event, but collect, disburse, and revoke do not emit program-level lifecycle events. These instructions can transfer tokens, mutate Record state, or close records, but observers must infer those actions by parsing transactions and token account deltas.

Impact

This does not create a direct fund-loss path because the affected instructions are manager/admin gated. The main impact is reduced monitoring and reconciliation quality. Missing events can slow incident response, complicate accounting, and make privileged operational mistakes harder to detect.

Likelihood

Low. The issue affects privileged operations and observability rather than direct authorization.

Recommendation

Emit events for collect, disburse, and revoke that include the relevant record, vault id or record key, amount, mint, source token account, destination token account, authority, and resulting record state.

Informational Issues

Unused manual account-close helper remains in the codebase

Resolved

Path

lib.rs

Function Name

close_account()

Description

The standalone close_account helper is not called by any Orbit instruction. It manually moves lamports, assigns the account to the System Program, and calls realloc(0, false). The active revoke path instead uses Anchor's close = admin account constraint.

Impact

No active exploit path was identified because the helper is unreachable. However, unused manual account-closing logic increases maintenance burden and currently contributes a deprecated realloc warning.

Likelihood

Low. The helper is dead code in the current implementation.

Recommendation

Remove the unused helper and the related SYSTEM_PROGRAM_ID import unless a future flow requires a documented manual close implementation.

Informational Issues

Unnecessary account requirements and writable metas add avoidable account surface

Resolved

Path

lib.rs

Function Name

collect(), disburse(), deposit()

Description

Collect and Disburse include system_program, but their handlers do not initialize, resize, close, assign, or transfer lamports through the System Program. Some signer and PDA authority accounts are also marked writable even though the handlers only use them as authorities or signer seeds.

Impact

This does not create a direct security issue. It adds avoidable account requirements, expands transaction write-lock sets, increases client-side friction, and makes the instruction interface noisier than necessary.

Likelihood

Low. This is an account-surface and maintainability issue.

Recommendation

Remove unused system_program accounts from Collect and Disburse unless future logic needs them. Remove unnecessary mut annotations from accounts that are not written by the handler.

Informational Issues

revoke can close any Orbit Record supplied by admin

Resolved

Path

lib.rs

Function Name

revoke()

Description

The revoke instruction is admin-gated, but its record account has no PDA seed constraint. Therefore, the configured admin can close any Orbit-owned Record, not only canonical whitelist records derived from the whitelist PDA seeds.

Impact

This is not a non-admin exploit because only the configured admin can call revoke. It remains an operator-safety issue if Record accounts are used for multiple domains, because a valid admin transaction can close an unintended record class.

Likelihood

Low. The issue requires admin authority.

Recommendation:

If revoke is intended only for whitelist records, add the relevant user_id and public_key instruction arguments and constrain the record with the same whitelist PDA seeds used by whitelist.

Automated Tests

No major issues were found. Some false positive errors were reported by the tools. All the other issues have been categorized above according to their level of severity.

Closing Summary

In this report, we have considered the security of ZynkLabs. We performed our audit according to the procedure described above.

No critical issues in ZynkLabs; all the issues mentioned above were resolved

Disclaimer

At QuillAudits, we have spent years helping projects strengthen their smart contract security. However, security is not a one-time event—threats evolve, and so do attack vectors. Our audit provides a security assessment based on the best industry practices at the time of review, identifying known vulnerabilities in the received smart contract source code.

This report does not serve as a security guarantee, investment advice, or an endorsement of any platform. It reflects our findings based on the provided code at the time of analysis and may no longer be relevant after any modifications. The presence of an audit does not imply that the contract is free of vulnerabilities or fully secure.

While we have conducted a thorough review, security is an ongoing process. We strongly recommend multiple independent audits, continuous monitoring, and a public bug bounty program to enhance resilience against emerging threats.

Stay proactive. Stay secure.

About QuillAudits

QuillAudits is a leading name in Web3 security, offering top-notch solutions to safeguard projects across DeFi, GameFi, NFT gaming, and all blockchain layers. With eight years of expertise, we've secured over 1500 projects globally, averting over $3 billion in losses. Our specialists rigorously audit smart contracts and ensure DApp safety on major platforms like Ethereum, BSC, Arbitrum, Algorand, Tron, Polygon, Polkadot, Fantom, NEAR, Solana, and others, guaranteeing your project's security with cutting-edge practices.

8+ Years of Expertise	**1M+** Lines of Code Audited
50+ Chains Supported	**1500+** Projects Secured

Follow Our Journey

AUDIT REPORT

June 2026

For

QuillAudits

Canada, India, Singapore, UAE, UK

www.quillaudits.com audits@quillaudits.com